



École Nationale d'Ingénieurs de Gabès
Département Génie Électrique et Automatique

Rapport de Projet

IA Gesture

Réalisé par :

Harathi Ilyes

Chihi Malek

Akkeri Farah

Fatnasi Arij

Chayeb May

Encadré par :

Mr. Fleh Aymen

Année universitaire : 2025-2026

Remerciements

Nous tenons à exprimer notre profonde gratitude à notre encadrant Mr Fleh Aymen pour son accompagnement, et son soutien tout au long de la réalisation de ce projet. Ses orientations pertinentes et son expertise nous ont permis de surmonter de nombreuses difficultés et d'améliorer la qualité de notre travail.

Nous remercions également l'ensemble des enseignants du département pour les connaissances et les compétences qu'ils nous ont transmises durant notre formation.

Nos sincères remerciements s'adressent aussi à nos familles et à nos proches pour leur soutien moral, leur patience et leurs encouragements constants.

Enfin, nous adressons nos remerciements à tous ceux qui ont contribué de près ou de loin à la réussite de ce projet.

Table des matières

1	Introduction Générale	7
1.1	Contexte général	7
1.2	Problématique	7
1.3	Objectifs	8
1.4	Présentation générale de la solution proposée	8
1.5	Structure du rapport	8
2	Étude et choix de la solution	10
2.1	Introduction	10
2.2	Première approche : utilisation de l'ESP32-CAM	10
2.3	Solutions alternatives étudiées	11
2.3.1	Raspberry Pi avec caméra USB	11
2.4	Solution retenue : caméra PC + traitement logiciel	12
2.5	Technologies logicielles utilisées	12
2.5.1	Vision par ordinateur et MediaPipe	12
2.5.2	Machine Learning	12
2.5.3	Communication MQTT	12
2.6	Comparaison des solutions étudiées	13
2.7	Synthèse	13
3	Analyse Fonctionnelle et Spécifications	14
3.1	Introduction	14
3.2	Analyse du besoin (Bête à cornes)	14
3.3	Analyse fonctionnelle externe (Diagramme pieuvre)	15
3.4	Cas d'utilisation du système	16
3.5	Spécifications fonctionnelles	17
3.6	Choix des composants matériels	18
3.7	Conclusion	18
4	Réalisation Logicielle – Intelligence Artificielle et Commu- nication IoT	20
4.1	Introduction du chapitre	20
4.2	Environnements de développement	20

4.3	Architecture logicielle globale	21
4.4	Acquisition et prétraitement des données visuelles	21
4.5	Détection de la main et extraction des landmarks	22
4.6	Construction du vecteur de caractéristiques	22
4.7	Reconnaissance des gestes par Machine Learning	22
4.8	Extrait de code principal – Prédiction et stabilisation	23
4.9	Algorithme de prédiction	23
4.10	Communication des résultats via MQTT	24
4.11	Discussion et limites	24
4.12	Conclusion	24
5	Réalisation Matérielle et Système Embarqué	25
5.1	Introduction	25
5.2	Architecture matérielle globale	25
5.2.1	Schéma fonctionnel du système	26
5.3	Choix du microcontrôleur	26
5.4	Périphériques matériels	26
5.5	Conception de la carte PCB	27
5.5.1	Objectifs de la conception	28
5.5.2	Placement des composants	28
5.5.3	Routage des pistes	28
5.6	Conception du boîtier	29
5.7	Résultat final	29
5.7.1	Principe de fonctionnement MQTT	29
5.8	Implémentation du firmware embarqué	30
5.8.1	Extrait de code principal (connexion MQTT)	30
5.9	Validation expérimentale	31
5.10	Limites et perspectives matérielles Limites	31
5.11	Conclusion	31
6	Perspectives et Améliorations	33
6.1	Introduction	33
6.2	Intégration de la caméra du smartphone	33
6.2.1	Motivation	33
6.2.2	Principe de fonctionnement	33
6.3	Amélioration du modèle d'intelligence artificielle	34
6.4	Sécurité avancée et détection de situations critiques	34
6.4.1	Détection de vol ou d'agression	34

6.4.2	Géolocalisation et suivi	34
6.5	Passage à une solution Cloud	34
6.6	Miniaturisation et conception d'un PCB final	35
6.7	Ouverture vers de nouveaux usages	35
6.8	Conclusion	35
A	Code source du système embarqué (ESP8266)	37
B	Script Python de reconnaissance gestuelle et communication MQTT	40

Table des figures

2.1	l'ESP32-CAM	10
2.2	Raspberry Pi avec caméra USB	11
3.1	Bête à cornes	15
3.2	Diagramme pieuvre	16
3.3	Cas d'utilisation du système	17
4.1	Environnements logiciels utilisés : PyCharm, MQTT et Ar- duino IDE	21
4.2	Architecture logicielle	21
4.3	Modèle de la main et points caractéristiques (MediaPipe) . . .	22
4.4	Description de votre image	24
5.1	Montage fonctionnel	26
5.2	Modélisation 3D du boîtier du système	28
5.3	Modélisation 3D du boîtier du système	29
5.4	Capture MQTT Explorer montrant les messages échangés . . .	31

Liste des tableaux

2.1	Tableau comparatif des différentes approches matérielles . . .	13
3.1	Spécifications fonctionnelles	18
3.2	Choix des composants matériels	18
5.1	Spécifications détaillées des composants matériels	27

Chapitre 1

Introduction Générale

1.1 Contexte général

La communication constitue un élément fondamental dans la vie humaine. Elle permet l'échange d'informations, l'expression des besoins, des émotions et des idées, et joue un rôle essentiel dans l'intégration sociale des individus. Grâce aux progrès technologiques rapides dans les domaines du numérique, de l'intelligence artificielle et de l'Internet des objets (IoT), de nouveaux moyens de communication intelligents ont vu le jour, facilitant l'interaction entre les personnes et leur environnement. Cependant, malgré ces avancées, certaines catégories de personnes, notamment les personnes en situation de handicap, continuent à rencontrer de grandes difficultés dans leur vie quotidienne. Les personnes non-verbales, malentendantes ou non-voyantes font face à des obstacles importants en matière de communication, de sécurité et d'autonomie.

1.2 Problématique

Les personnes non-verbales et sourdes utilisent principalement le langage des signes pour communiquer. Or, ce langage n'est pas compris par la majorité de la population, ce qui limite fortement leurs interactions sociales. De même, les personnes non-voyantes sont souvent exposées à des situations de danger telles que les obstacles physiques ou les tentatives de vol, sans disposer de moyens efficaces pour les détecter rapidement. Ces constats ont motivé la réalisation de ce projet, dont l'objectif est de proposer une solution technologique intelligente permettant d'assister les personnes en situation de handicap, en améliorant leur communication, leur sécurité et leur autonomie.

1.3 Objectifs

L'objectif principal de ce projet est de concevoir et de développer un système intelligent capable de traduire des gestes de la main en messages compréhensibles, afin d'aider les personnes non-verbales et malentendantes à communiquer plus facilement avec leur entourage. Les objectifs spécifiques du projet sont les suivants :

- Capturer les gestes de la main à l'aide d'une caméra.
- Utiliser des techniques de vision par ordinateur et d'intelligence artificielle pour reconnaître les gestes.
- Transmettre les résultats du traitement vers un système embarqué.
- Afficher les messages traduits sur un écran OLED.
- Améliorer la sécurité des utilisateurs grâce à des capteurs et des alertes sonores.

1.4 Présentation générale de la solution proposée

La solution proposée repose sur une architecture hybride combinant un système de vision par ordinateur et un module embarqué. Une caméra, initialement celle d'un PC et ultérieurement celle d'un smartphone, est utilisée pour capturer les gestes de la main. Les images sont traitées à l'aide d'algorithmes d'intelligence artificielle afin d'identifier les gestes correspondants à des mots prédéfinis. Les résultats de la reconnaissance sont transmis via un protocole de communication léger (MQTT) vers un module ESP32, chargé d'afficher les messages traduits sur un écran OLED. Un capteur ultrasonique est intégré afin d'assurer une distance optimale entre l'utilisateur et la caméra, améliorant ainsi la précision de la reconnaissance. Un buzzer est également utilisé pour signaler les situations de danger ou d'urgence.

1.5 Structure du rapport

Ce rapport est structuré comme suit :

- Chapitre 2 Étude des Solutions Existantes et Choix Technologiques.
- Chapitre 3 : Analyse Fonctionnelle et Spécifications.

- Chapitre 4 est consacré à la réalisation logicielle et aux aspects liés à l'intelligence artificielle.
- Chapitre 5 présente la réalisation matérielle et embarquée du système.
- Chapitre 6 propose des perspectives d'amélioration et des évolutions futures du projet.

Enfin, une conclusion générale récapitule le travail réalisé et ouvre sur les perspectives futures.

Chapitre 2

Étude et choix de la solution

2.1 Introduction

Ce chapitre présente les différentes solutions techniques étudiées au début du projet ainsi que les difficultés rencontrées lors de leur mise en œuvre. Cette analyse comparative a permis d'orienter le choix final vers une solution plus adaptée, fiable et économiquement accessible, en adéquation avec les objectifs du projet.

2.2 Première approche : utilisation de l'ESP32-CAM

Dans une première phase, l'ESP32-CAM a été envisagée comme solution principale pour la capture d'images et la reconnaissance des gestes. Ce choix était motivé par son faible coût, son format compact et l'intégration directe d'une caméra avec un microcontrôleur.

Cependant, plusieurs problèmes techniques ont été rencontrés :



FIGURE 2.1 – l'ESP32-CAM

- Difficultés fréquentes de téléversement du programme (problèmes liés au mode BOOT et RESET).
 - Instabilité de la communication série.
 - Capacité de traitement limitée pour exécuter des algorithmes de vision par ordinateur et d'intelligence artificielle.
 - Qualité d'image insuffisante pour une reconnaissance fiable des gestes.
- Ces contraintes ont rendu l'utilisation de l'ESP32-CAM peu adaptée aux exigences du projet.

2.3 Solutions alternatives étudiées

2.3.1 Raspberry Pi avec caméra USB

Une autre solution étudiée consistait à utiliser un Raspberry Pi associé à une caméra USB. Cette plateforme offre une puissance de calcul suffisante pour exécuter des algorithmes avancés de vision par ordinateur et de Machine Learning.



FIGURE 2.2 – Raspberry Pi avec caméra USB

Néanmoins, cette solution présente des inconvénients majeurs :

- Coût élevé du matériel.
- Consommation énergétique importante.
- Complexité de configuration et de maintenance.

Ces facteurs rendent cette solution moins accessible pour une application destinée au grand public.

2.4 Solution retenue : caméra PC + traitement logiciel

Face aux limitations des solutions précédentes, le choix s'est porté sur l'utilisation d'une caméra PC couplée à un traitement logiciel exécuté sur ordinateur. Cette approche présente plusieurs avantages :

- Puissance de calcul suffisante pour exécuter MediaPipe et les modèles de Machine Learning.
- Stabilité du système et facilité de débogage.
- Réduction significative des coûts.
- Possibilité d'évolution vers l'utilisation d'une caméra de smartphone.

Les images capturées par la caméra sont traitées localement, puis les résultats de reconnaissance sont transmis à un module ESP32 via le protocole MQTT.

2.5 Technologies logicielles utilisées

2.5.1 Vision par ordinateur et MediaPipe

La bibliothèque MediaPipe est utilisée pour détecter et extraire les points clés (landmarks) de la main en temps réel. Ces données constituent la base du processus de reconnaissance des gestes.

2.5.2 Machine Learning

À partir des landmarks extraits, un modèle de Machine Learning supervisé est entraîné afin de classifier les gestes et les associer à des mots prédéfinis (ex. : Hello, Yes, No, Thanks).

2.5.3 Communication MQTT

Le protocole MQTT est utilisé pour assurer une communication fiable et rapide entre la partie logicielle (PC ou smartphone) et la partie embarquée (ESP32).

2.6 Comparaison des solutions étudiées

Afin de choisir la solution technique la plus adaptée aux contraintes de notre projet, nous avons réalisé une étude comparative. Cette analyse prend en compte des critères essentiels tels que le coût, la performance de calcul et la facilité de mise en œuvre. Le tableau suivant synthétise les points forts et les limitations des trois approches envisagées.

Solution	Avantages	Inconvénients
ESP32-CAM	Faible coût, compact	Faible puissance, instabilité
Raspberry Pi + caméra	Haute performance	Coût élevé, complexité
Caméra PC + PC	Stable, performant, économique	Dépendance à un PC

TABLE 2.1 – Tableau comparatif des différentes approches matérielles

2.7 Synthèse

Cette étude comparative a permis de justifier le choix d’une architecture hybride basée sur une caméra PC et un module ESP32. Cette solution offre un bon compromis entre performance, coût et évolutivité, tout en répondant efficacement aux objectifs du projet. Des figures illustrant l’architecture globale et le flux de données seront présentées dans le chapitre suivant.

Chapitre 3

Analyse Fonctionnelle et Spécifications

3.1 Introduction

Ce chapitre a pour objectif d'analyser le fonctionnement global du système à travers une approche fonctionnelle. Cette analyse permet d'identifier clairement les besoins des utilisateurs, les fonctions principales du système ainsi que les contraintes techniques, afin de définir des spécifications cohérentes et adaptées au projet.

3.2 Analyse du besoin (Bête à cornes)

L'analyse du besoin constitue une étape essentielle dans la conception d'un système, car elle permet de définir clairement à quoi sert le produit, pour qui il est conçu et dans quel objectif il est développé. Cette analyse est généralement représentée par l'outil méthodologique appelé Bête à cornes. Dans le cadre de notre projet, le système rend service principalement aux personnes en situation de handicap, en particulier les personnes non-verbales, malentendantes et non-voyantes. Ces utilisateurs rencontrent quotidiennement des difficultés de communication et de sécurité, ce qui limite leur autonomie et leur intégration sociale. Le système agit principalement sur deux aspects fondamentaux :

- la communication, en traduisant les gestes de la main en messages compréhensibles.
- la sécurité, en alertant l'utilisateur en cas de danger ou de situation anormale.

La finalité du système est donc d'améliorer la qualité de vie des personnes en situation de handicap en leur offrant un moyen de communication

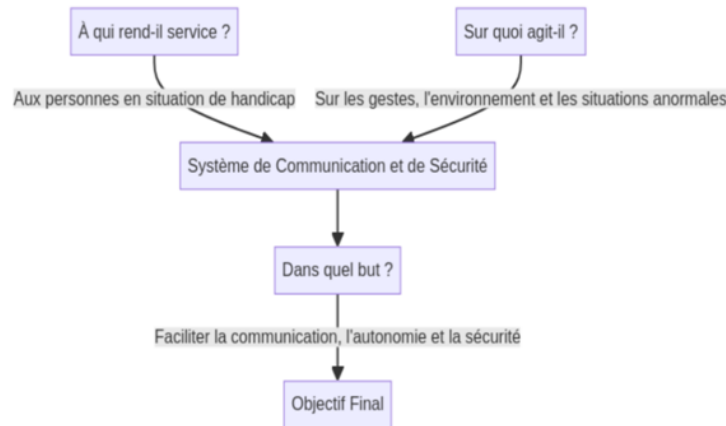


FIGURE 3.1 – Bête à cornes

accessible et un dispositif d'assistance intelligent.

Bête à cornes (description détaillée) :

- À qui rend service le système ? Aux personnes en situation de handicap (non-verbales, sourdes, non-voyantes).
- Sur quoi agit-il ? Sur la communication gestuelle et la sécurité de l'utilisateur.
- Dans quel but ? Faciliter la communication, renforcer l'autonomie et assurer la protection de l'utilisateur.

3.3 Analyse fonctionnelle externe (Diagramme pieuvre)

Le diagramme pieuvre permet d'identifier les interactions entre le système et les éléments de son environnement externe. Il met en évidence les fonctions de service que le système doit assurer ainsi que les contraintes auxquelles il est soumis. Dans notre projet, le système interagit avec plusieurs acteurs et éléments externes, notamment : l'utilisateur, la caméra (PC ou smartphone), le réseau Internet, le module ESP32, les dispositifs d'alerte (écran OLED, buzzer).

MQTT a été choisi pour : Sa faible latence, son faible coût énergétique...

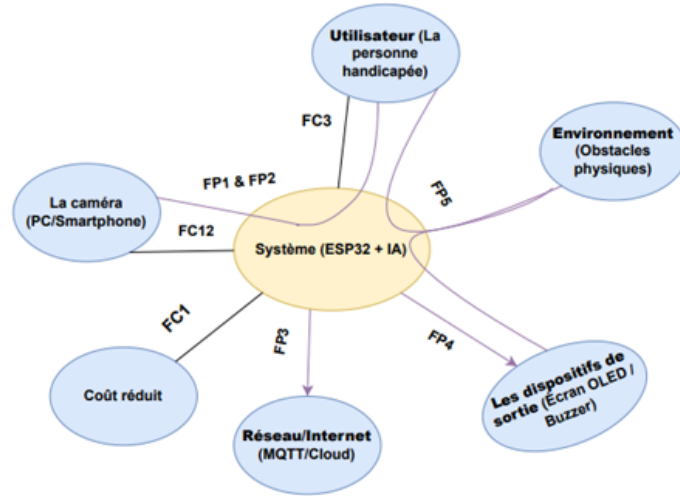


FIGURE 3.2 – Diagramme pieuvre

Les principales fonctions de service identifiées sont les suivantes :

- FP1 : Capturer les gestes de la main à l'aide de la caméra.
- FP2 : Reconnaître et interpréter les gestes grâce aux algorithmes de vision par ordinateur et de Machine Learning.
- FP3 : Transmettre les résultats vers le système embarqué via le réseau.
- FP4 : Afficher les messages traduits sur l'écran OLED.
- FP5 : Alerter l'utilisateur en cas de danger ou de situation anormale.

Les contraintes principales du système sont :

- C1 : Temps de réponse réduit pour une utilisation en temps réel.
- C2 : Simplicité d'utilisation pour des utilisateurs non techniques.
- C3 : Coût réduit afin de rendre la solution accessible.

3.4 Cas d'utilisation du système

Les cas d'utilisation permettent de décrire le fonctionnement du système du point de vue de l'utilisateur. Ils illustrent les différentes interactions possibles entre l'utilisateur et le système.

Le scénario principal de fonctionnement est le suivant :

1. L'utilisateur se place devant la caméra et effectue un geste de la main.
2. La caméra capture les images en temps réel.

3. Le système de vision par ordinateur détecte la main et extrait les points clés.
4. Le modèle de Machine Learning reconnaît le geste correspondant.
5. Le message associé est envoyé au module ESP32 via MQTT.
6. Le message est affiché sur l'écran OLED.
7. En cas de danger ou d'urgence, une alerte sonore est déclenchée.

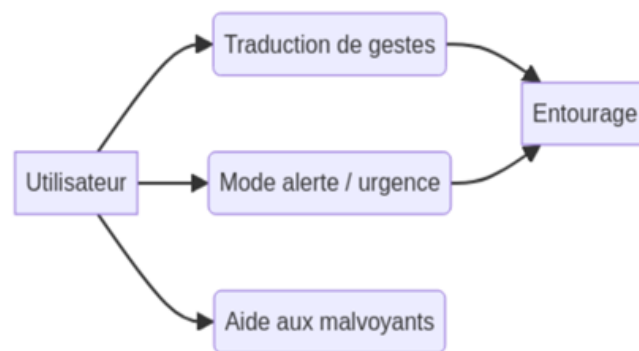


FIGURE 3.3 – Cas d'utilisation du système

Ce scénario montre que le système fonctionne de manière transparente pour l'utilisateur, sans nécessiter d'interaction complexe.

3.5 Spécifications fonctionnelles

Le tableau suivant présente les exigences fonctionnelles du système ainsi que les critères permettant de valider leur bon fonctionnement :

Fonction	Description	Critère
Acquisition	Capture vidéo des gestes	Caméra fonctionnelle
Reconnaissance	Classification des gestes	Taux de reconnaissance
Communication	Transmission des données	Latence faible
Restitution	Affichage et alerte	Lisibilité et audibilité

TABLE 3.1 – Spécifications fonctionnelles

3.6 Choix des composants matériels

Le choix des composants matériels est une étape cruciale pour garantir l'efficacité et la viabilité du système. Le tableau ci-dessous récapitule les solutions retenues ainsi que les justifications techniques associées :

Élément	Choix	Justification
Caméra	PC / Smartphone	Accessibilité et coût réduit
Traitement	Python + MediaPipe	Haute performance et grande flexibilité
Communication	MQTT	Protocole léger et transmission fiable
Microcontrôleur	ESP32	Excellente connectivité (Wi-Fi/BT) et puissance
Interface	OLED + LEDs + Buzzer	Simplicité d'utilisation et efficacité des alertes

TABLE 3.2 – Choix des composants matériels

3.7 Conclusion

L'analyse fonctionnelle réalisée dans ce chapitre a permis de clarifier les besoins, les fonctions et les contraintes du système. Ces éléments constituent

la base des choix architecturaux et techniques présentés dans le chapitre suivant consacré à la conception de la solution.

Chapitre 4

Réalisation Logicielle – Intelligence Artificielle et Communication IoT

4.1 Introduction du chapitre

Ce chapitre est consacré à la réalisation logicielle du système proposé. Il décrit les méthodes et outils utilisés pour assurer la reconnaissance automatique des gestes de la main, l'analyse intelligente des données visuelles, ainsi que la transmission des résultats vers un système embarqué via une infrastructure IoT. L'objectif principal est de concevoir une solution fiable, modulaire et évolutive, capable de fonctionner en temps réel tout en respectant les contraintes de performance et de précision.

4.2 Environnements de développement

Dans ce projet, plusieurs environnements logiciels ont été utilisés pour assurer le développement et la communication du système. PyCharm est employé pour l'implémentation des algorithmes de reconnaissance des gestes en Python. Arduino IDE permet la programmation de la carte ESP32 et la gestion des périphériques. Le protocole MQTT assure l'échange des données entre la partie logicielle et le système embarqué.



FIGURE 4.1 – Environnements logiciels utilisés : PyCharm, MQTT et Arduino IDE

4.3 Architecture logicielle globale

La solution logicielle repose sur une architecture client/serveur distribuée, dans laquelle le traitement intensif est effectué côté PC, tandis que l’affichage et les interactions physiques sont gérés par un système embarqué.

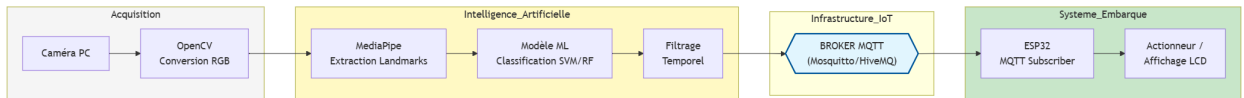


FIGURE 4.2 – Architecture logicielle

4.4 Acquisition et prétraitement des données visuelles

L’acquisition vidéo est réalisée à l’aide de la bibliothèque OpenCV, qui permet la capture en temps réel des images provenant de la caméra du PC. Les images sont converties du format BGR au format RGB afin d’être compatibles avec le moteur de traitement MediaPipe. Cette étape garantit une détection plus précise des structures de la main.

4.5 Détection de la main et extraction des landmarks

La bibliothèque MediaPipe Hands est utilisée pour détecter automatiquement la main dans chaque image. Elle fournit un modèle optimisé basé sur des réseaux de neurones convolutifs, capable de fonctionner en temps réel. Chaque main détectée est décrite par 21 landmarks tridimensionnels, représentant les articulations et les extrémités des doigts.

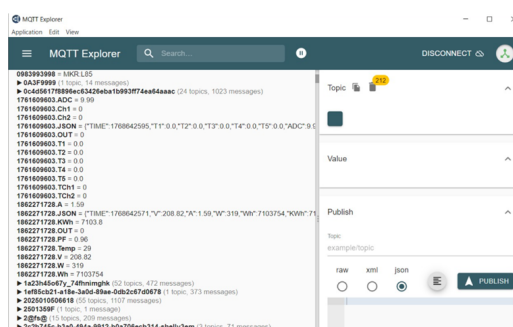


FIGURE 4.3 – Modèle de la main et points caractéristiques (MediaPipe)

4.6 Construction du vecteur de caractéristiques

Les coordonnées (x,y,z) de chaque landmark sont concaténées pour former un vecteur de 63 caractéristiques, utilisé comme entrée du modèle de Machine Learning. Cette représentation permet :

- une invariance partielle à l'échelle.
- une réduction de la dépendance à l'orientation.
- une bonne généralisation du modèle.

4.7 Reconnaissance des gestes par Machine Learning

Le modèle d'apprentissage automatique est entraîné hors ligne à partir d'un ensemble de gestes prédéfinis correspondant à des mots ou des commandes simples. Lors de l'exécution :

- le modèle calcule les probabilités associées à chaque classe.
- la classe la plus probable est sélectionnée.
- une vérification temporelle est appliquée pour réduire les erreurs.

4.8 Extrait de code principal – Prédiction et stabilisation

L'extrait suivant illustre la partie centrale du traitement intelligent, combinant classification et filtrage temporel.

4.9 Algorithme de prédiction

Le fragment de code suivant illustre le processus de prédiction du geste et le mécanisme de lissage par historique :

```
# Calcul des probabilités de prédiction
proba = model.predict_proba([points])[0]
gesture = model.classes_[np.argmax(proba)]

# Gestion de l'historique pour stabiliser la prédiction
pred_history.append(gesture)
if len(pred_history) > 5:
    pred_history.pop(0)

# Sélection du geste le plus fréquent (vote majoritaire)
gesture = max(set(pred_history), key=pred_history.count)
```

Listing 4.1 – Script de prédiction et gestion de l'historique

Analyse technique

- La fonction `predict_proba` permet d'obtenir une mesure de confiance.
- Le filtrage par vote majoritaire augmente la stabilité du système.
- Cette stratégie est particulièrement adaptée aux environnements réels bruités.

4.10 Communication des résultats via MQTT

Une fois le geste reconnu, le résultat est transmis au module embarqué à l'aide du protocole MQTT. MQTT a été choisi pour : Sa faible latence, son faible coût énergétique, son adaptation aux systèmes IoT distribués.

MQTT a été choisi pour : Sa faible latence, son faible coût énergétique...

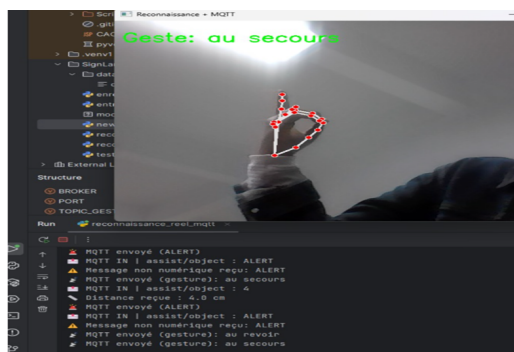


FIGURE 4.4 – Description de votre image

4.11 Discussion et limites

Malgré de bonnes performances, certaines limites subsistent :

- dépendance à la qualité de l'éclairage.
- sensibilité aux gestes très rapides.
- nécessité d'un apprentissage préalable du modèle.

Ces limites ouvrent la voie à des améliorations futures telles que l'intégration de réseaux de neurones plus complexes ou l'utilisation de capteurs supplémentaires.

4.12 Conclusion

Ce chapitre a présenté la réalisation logicielle complète du système, depuis l'acquisition des données visuelles jusqu'à la communication des résultats via MQTT. Cette approche modulaire constitue une base solide pour l'extension du projet vers des plateformes mobiles ou des services Cloud.

Chapitre 5

Réalisation Matérielle et Système Embarqué

5.1 Introduction

Après la conception logicielle et l'architecture fonctionnelle du système, ce chapitre est consacré à la réalisation matérielle du projet et à la mise en œuvre du système embarqué. L'objectif principal est d'assurer une communication fiable entre les différents composants matériels (capteurs, microcontrôleur, actionneurs et interfaces d'affichage) tout en garantissant une intégration cohérente avec la partie logicielle basée sur la communication MQTT. Le système embarqué joue un rôle central dans le projet, puisqu'il constitue l'interface entre le traitement des données effectué sur le PC (ou smartphone à l'avenir) et l'utilisateur final, notamment les personnes en situation de handicap.

5.2 Architecture matérielle globale

L'architecture matérielle du système repose sur une structure modulaire permettant une évolution future du projet. Elle est composée principalement des éléments suivants :

- Un microcontrôleur ESP32
- Un écran OLED SSD1306
- Un capteur ultrasonique
- Un buzzer d'alerte
- Une alimentation adaptée
- Un module de communication Wi-Fi intégré

5.2.1 Schéma fonctionnel du système

Le fonctionnement global peut être résumé comme suit :

1. Le PC capture les informations via la caméra et traite les données.
2. Les résultats sont envoyés au cloud via le protocole MQTT.
3. L'ESP32 reçoit les messages MQTT
4. Les données sont affichées sur l'écran OLED.
5. Les alertes sont signalées via le buzzer.
6. Le capteur ultrasonique assure la sécurité et l'optimisation de la distance.

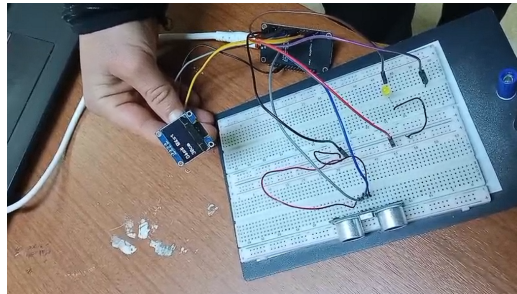


FIGURE 5.1 – Montage fonctionnel

5.3 Choix du microcontrôleur

Le microcontrôleur ESP32 a été choisi pour les raisons suivantes :

- Intégration native du Wi-Fi
- Faible consommation énergétique
- Large communauté et support logiciel
- Compatibilité avec MQTT
- Facilité d'intégration avec capteurs et afficheurs

Ce choix permet une communication directe avec le serveur MQTT sans nécessiter de modules externes supplémentaires.

5.4 Périphériques matériels

Afin d'assurer la cohérence technique du prototype, le choix de chaque composant a été rigoureusement analysé en fonction de son rôle spécifique,

de son interface de communication et de sa valeur ajoutée pour l'utilisateur final.

Composant	Référence	Interface	Rôle	Justification
Écran OLED	SSD1306	I ² C	Affichage des messages, états et alertes	Faible consommation, lisibilité, compact
Capteur ultrason	HC-SR04	GPIO	Mesure de distance et assistance	Précis, faible coût, standard
Buzzer	Actif	GPIO	Alertes sonores de danger	Simple à piloter, retour immédiat
Microcontrôleur	ESP32	Wi-Fi/I ² C	Traitement, MQTT et gestion capteurs	Wi-Fi intégré, support MQTT
Module Wi-Fi	Intégré	Wi-Fi	Communication broker MQTT	Fiabilité, pas de matériel externe
Alimentation	5V / USB	—	Alimentation du système	Disponible, compatible

TABLE 5.1 – Spécifications détaillées des composants matériels

5.5 Conception de la carte PCB

La conception de la carte PCB vise à regrouper l'ensemble des composants sur une seule carte afin de remplacer le montage sur breadboard, souvent instable. Cette étape permet d'améliorer la fiabilité, de réduire les erreurs de

câblage et de faciliter l'intégration finale dans un boîtier.

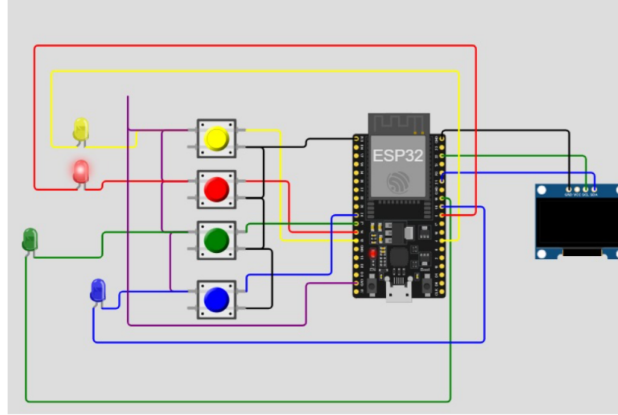


FIGURE 5.2 – Modélisation 3D du boîtier du système

5.5.1 Objectifs de la conception

Les principaux objectifs sont :

- Réduction de l'encombrement.
- Amélioration de la fiabilité électrique.
- Simplification du câblage.
- Facilitation de la maintenance.

5.5.2 Placement des composants

Les composants sont placés de manière stratégique afin de réduire la longueur des pistes et faciliter les connexions. L'ESP32 est positionné au centre de la carte, tandis que les boutons sont placés en façade pour permettre une interaction simple avec l'utilisateur.

5.5.3 Routage des pistes

Le routage consiste à tracer les pistes reliant les composants entre eux. Une attention particulière est portée aux pistes d'alimentation qui sont réalisées plus larges afin d'assurer une bonne distribution de l'énergie.

5.6 Conception du boîtier

Un boîtier a été conçu pour protéger la carte PCB et assurer une bonne ergonomie.

Des ouvertures sont prévues pour l'écran OLED, les boutons, les LEDs et les ports de connexion.

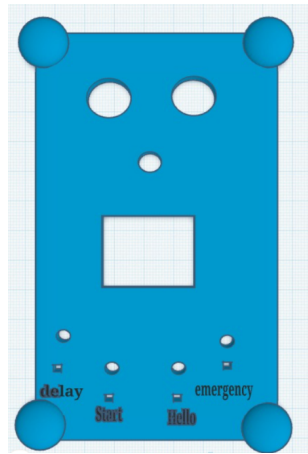


FIGURE 5.3 – Modélisation 3D du boîtier du système

5.7 Résultat final

La réalisation matérielle aboutit à un système compact et fonctionnel intégrant l'ensemble des composants sur une carte PCB unique. Le système obtenu est fiable, simple d'utilisation et prêt pour une utilisation pratique. La communication entre le PC et le système embarqué repose sur le protocole MQTT, reconnu pour sa légèreté et son efficacité dans les systèmes IoT.

5.7.1 Principe de fonctionnement MQTT

- Le PC joue le rôle de publisher.
- L'ESP32 agit comme subscriber.
- Le broker MQTT centralise les échanges.

5.8 Implémentation du firmware embarqué

Le firmware embarqué est développé sous Arduino IDE. Il assure les fonctions suivantes :

- Connexion au réseau Wi-Fi.
- Connexion au broker MQTT.
- Réception des messages.
- Affichage sur OLED.
- Gestion des capteurs et actionneurs.

5.8.1 Extrait de code principal (connexion MQTT)

```
void setup() {
  Serial.begin(115200);

  // Initialisation de l'écran OLED a l'adresse 0x3C
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED init failed");
    while (true); // Arrêt en cas d'échec
  }

  setup_wifi(); // Connexion au reseau sans fil
  client.setServer(mqtt_server, mqtt_port); //
    Configuration du broker MQTT
  client.setCallback(callback); // Definition de la
    fonction de reception
}
```

Listing 5.1 – Initialisation du système (OLED, WiFi et MQTT)

Explication :

- Initialisation de la communication série.
- Initialisation de l'écran OLED.
- Connexion au réseau Wi-Fi.
- Configuration du serveur MQTT.
- Définition de la fonction de rappel pour la réception des messages.

5.9 Validation expérimentale

La validation du système a été réalisée à l'aide de :

- MQTT Explorer pour visualiser les messages échangés.
- Moniteur série Arduino pour le débogage.
- Tests fonctionnels sur chaque composant matériel.

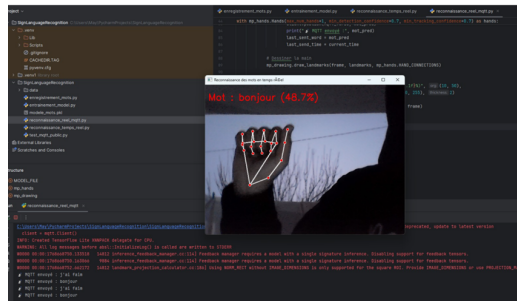


FIGURE 5.4 – Capture MQTT Explorer montrant les messages échangés

Les résultats montrent :

- Une communication stable.
- Un affichage correct des messages.
- Une réponse rapide du système aux événements.

5.10 Limites et perspectives matérielles Limites

- Dépendance à une connexion Internet.
- Utilisation actuelle d'une caméra PC.
- Précision limitée par l'environnement lumineux Perspectives.
- Intégration directe de la caméra smartphone.
- Conception d'une carte PCB dédiée.
- Ajout de modules GPS et sécurité avancée.
- Optimisation énergétique.

5.11 Conclusion

Ce chapitre a présenté la réalisation matérielle complète du système et l'intégration du système embarqué. Les choix effectués permettent une solu-

tion modulaire, évolutive et accessible, répondant aux besoins des personnes en situation de handicap. La cohérence entre le matériel et le logiciel garantit une base solide pour les développements futurs du projet.

Chapitre 6

Perspectives et Améliorations

6.1 Introduction

Bien que les résultats obtenus soient satisfaisants et démontrent la faisabilité du système proposé, plusieurs pistes d'amélioration peuvent être envisagées afin d'augmenter les performances, la portabilité et l'accessibilité de la solution. Ce chapitre présente les évolutions possibles à court, moyen et long terme, aussi bien sur le plan logiciel que matériel.

6.2 Intégration de la caméra du smartphone

6.2.1 Motivation

L'utilisation d'un PC équipé d'une webcam, bien que fonctionnelle, limite la mobilité du système. L'intégration d'une caméra de smartphone permettrait :

- une meilleure accessibilité.
- une portabilité accrue.
- une réduction du coût global.
- une utilisation plus naturelle pour les utilisateurs finaux.

6.2.2 Principe de fonctionnement

Deux approches sont envisageables :

- Streaming vidéo du smartphone vers le PC ou le cloud.

Le smartphone capture la vidéo et l'envoie via Wi-Fi. Le traitement reste assuré par le modèle existant.

- Traitement embarqué directement sur le smartphone.

Utilisation d'une application mobile intégrant MediaPipe et le modèle ML, avec envoi des résultats via MQTT. Cette évolution permettrait de supprimer totalement la dépendance au PC.

6.3 Amélioration du modèle d'intelligence artificielle

Plusieurs axes peuvent améliorer la reconnaissance des gestes :

- augmentation de la base de données d'apprentissage.
- ajout de gestes dynamiques (mouvements).
- utilisation de modèles plus avancés (réseaux neuronaux profonds).
- adaptation du modèle aux différents utilisateurs.

Ces améliorations augmenteraient la précision et la robustesse du système.

6.4 Sécurité avancée et détection de situations critiques

6.4.1 Détection de vol ou d'agression

Le système peut être enrichi par :

- la détection automatique de comportements suspects.
- l'identification de visages ou d'objets dangereux.
- l'activation automatique des alertes sonores et visuelles.

6.4.2 Géolocalisation et suivi

L'ajout d'un module GPS permettrait :

- le suivi de la position de l'utilisateur.
- l'envoi de notifications à un proche ou à un service d'assistance.
- une meilleure protection des personnes vulnérables.

6.5 Passage à une solution Cloud

Le déploiement du traitement dans le cloud offrirait :

- une puissance de calcul plus élevée.

- des mises à jour du modèle à distance.
- un accès multi-utilisateurs.
- une meilleure scalabilité du système.

Le protocole MQTT reste parfaitement compatible avec cette évolution.

6.6 Miniaturisation et conception d'un PCB final

Une amélioration importante concerne la réalisation d'une carte PCB dédiée, intégrant :

- le microcontrôleur
- les capteurs.
- l'écran OLED.
- les actionneurs (buzzer, LED).

Cela permettrait :

- une meilleure fiabilité.
- une réduction de la taille.
- une intégration dans un boîtier ergonomique.

6.7 Ouverture vers de nouveaux usages

Le projet peut être étendu à d'autres domaines :

- assistance aux personnes âgées.
- systèmes éducatifs interactifs.
- dispositifs de sécurité intelligents.
- interfaces homme-machine naturelles.

6.8 Conclusion

Ce chapitre a mis en évidence que le projet développé constitue une base solide et évolutive. Les perspectives proposées ouvrent la voie vers une solution complète, intelligente et accessible, capable de répondre aux besoins réels des personnes en situation de handicap.

Conclusion Générale

En conclusion, ce chapitre a permis de consolider les notions fondamentales abordées tout au long du cours et de mieux comprendre le rôle des concepts étudiés dans l'analyse et la modélisation des systèmes. Les différentes méthodes présentées ont montré leur importance aussi bien sur le plan théorique que pratique, en mettant en évidence leur complémentarité et leurs domaines d'application. L'étude détaillée des principes, accompagnée d'exemples et d'interprétations, permet de mieux maîtriser les mécanismes de fonctionnement et d'identifier les paramètres clés influençant les performances du système. Ce chapitre constitue ainsi une étape essentielle pour la compréhension globale du module et prépare efficacement à l'étude de situations plus complexes. Enfin, les connaissances acquises dans ce chapitre sont indispensables pour la suite du cours et pour leur application dans des contextes réels, notamment en ingénierie et en automatisation, où la rigueur d'analyse et la maîtrise des outils sont primordiales.

Annexe A

Code source du système embarqué (ESP8266)

Ce code assure la gestion des capteurs ultrasoniques, de l'affichage OLED, ainsi que la communication bidirectionnelle via le protocole MQTT.

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// ===== Wi-Fi credentials =====
const char* ssid = "Redmi 10";
const char* password = "11111111";

// ===== MQTT broker & topics =====
const char* mqtt_server = "broker.emqx.io";
const int mqtt_port = 1883;
const char* topic_gesture = "assist/gesture";
const char* topic_object = "assist/object";

// ===== OLED setup =====
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// ===== Ultrason pins (GPIO) =====
#define TRIG_PIN 14 // D5
#define ECHO_PIN 12 // D6

// ===== LED + Buzzer pins =====
#define LED_PIN 13 // D7
#define BUZZER_PIN 15 // D8

WiFiClient espClient;
PubSubClient client(espClient);
unsigned long lastPublishMs = 0;
const unsigned long publishIntervalMs = 1000;

// ===== OLED helper =====
void showText(const String& line1, const String& line2 = "") {
  display.clearDisplay();
  display.setTextSize(2);
```

```

    display.setTextColor(SSD1306_WHITE);
    display.setCursor(0, 8);
    display.println(line1);
    if (line2.length()) {
        display.setCursor(0, 36);
        display.println(line2);
    }
    display.display();
}

// ===== Ultrasonic distance =====
long getDistanceCm() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);
    long duration = pulseIn(ECHO_PIN, HIGH, 25000);
    if (duration == 0) return -1;
    return duration * 0.034 / 2;
}

// ===== WiFi =====
void setup_wifi() {
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);
    showText("WiFi...", "Connecting");
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    showText("WiFi OK", WiFi.localIP().toString());
    delay(800);
}

// ===== MQTT callback =====
void callback(char* topic, byte* payload, unsigned int length) {
    String msg;
    for (unsigned int i = 0; i < length; i++) msg += (char)payload[i];

    if (String(topic) == topic_gesture) {
        showText("Gesture", msg);
    }
    else if (String(topic) == topic_object) {
        if (msg == "STOLEN" || msg == "DANGER") {
            showText("ALERTE", msg);
            digitalWrite(LED_PIN, HIGH);
            digitalWrite(BUZZER_PIN, HIGH);
        }
        else {
            long dist = msg.toInt();
            if (dist > 0) {
                showText("Dist RX", String(dist) + "cm");
                digitalWrite(LED_PIN, LOW);
                digitalWrite(BUZZER_PIN, LOW);
            }
        }
    }
}

```

```

}

// ===== MQTT reconnect =====
void reconnect() {
    while (!client.connected()) {
        String clientId = "ESP8266_" + String(ESP.getChipId(), HEX);
        if (client.connect(clientId.c_str())) {
            client.subscribe(topic_gesture);
            client.subscribe(topic_object);
            showText("MQTT OK", "Subscribed");
        } else {
            delay(3000);
        }
    }
}

void setup() {
    Serial.begin(115200);
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        while (true) delay(1);
    }
    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);
    pinMode(LED_PIN, OUTPUT);
    pinMode(BUZZER_PIN, OUTPUT);

    setup_wifi();
    client.setServer(mqtt_server, mqtt_port);
    client.setCallback(callback);
}

void loop() {
    if (!client.connected()) reconnect();
    client.loop();

    long distance = getDistanceCm();
    if (distance > 0) {
        unsigned long now = millis();
        if (now - lastPublishMs >= publishIntervalMs) {
            String distMsg = String(distance);
            client.publish(topic_object, distMsg.c_str());
            showText("Send Dist", distMsg + "cm");
            lastPublishMs = now;
        }
    }
    delay(150);
}

```

Listing A.1 – Programme complet du microcontrôleur

Annexe B

Script Python de reconnaissance gestuelle et communication MQTT

Ce script Python constitue le cœur de l'intelligence artificielle du projet. Il utilise MediaPipe pour la détection des points de la main, un modèle Machine Learning chargé via Pickle pour la classification, et la bibliothèque Paho-MQTT pour échanger des données en temps réel avec le microcontrôleur.

```
import cv2
import mediapipe as mp
import pickle
import numpy as np
import os
import time
import paho.mqtt.client as mqtt

# ===== MQTT CONFIG =====
BROKER = "broker.emqx.io"
PORT = 1883
TOPIC_GESTURE = "assist/gesture" # PC -> ESP (gestes)
TOPIC_OBJECT = "assist/object" # ESP -> PC (distance) + PC -> ESP (
    alertes)

# ===== Verification du modele =====
MODEL_FILE = "modele_mots.pkl"
if not os.path.exists(MODEL_FILE):
    raise SystemExit("Le modele n'existe pas.")

with open(MODEL_FILE, "rb") as f:
    model = pickle.load(f)

# ===== MediaPipe =====
mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils

# ===== Callback MQTT (Reception distance) =====
def on_message(client, userdata, msg):
    payload = msg.payload.decode()
    if msg.topic == TOPIC_OBJECT:
        try:
            distance = float(payload)
            # Logique d'alerte de proximite
            if distance < 30:
```

```

        client.publish(TOPIC_OBJECT, "ALERT")
        print("ALERT envoy : obstacle detecte")
    except ValueError:
        pass

# ===== MQTT client setup =====
client = mqtt.Client(protocol=mqtt.MQTTv311)
client.on_message = on_message
client.connect(BROKER, PORT, 60)
client.loop_start()
client.subscribe(TOPIC_OBJECT)

last_sent = ""
last_time = 0
SEND_INTERVAL = 1.0
pred_history = []
HISTORY_LENGTH = 5

# ===== Flux Video =====
cap = cv2.VideoCapture(0)

with mp_hands.Hands(max_num_hands=1,
                    min_detection_confidence=0.7,
                    min_tracking_confidence=0.7) as hands:

    while True:
        ret, frame = cap.read()
        if not ret: break

        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = hands.process(frame_rgb)
        frame = cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2BGR)

        mot_pred = ""

        if results.multi_hand_landmarks:
            landmarks = results.multi_hand_landmarks[0]
            points = []
            for lm in landmarks.landmark:
                points.extend([lm.x, lm.y, lm.z])

            try:
                proba = model.predict_proba([points])[0]
                mot_pred = model.classes_[np.argmax(proba)]
            except:
                pass

            # Lissage de la prediction par historique
            pred_history.append(mot_pred)
            if len(pred_history) > HISTORY_LENGTH:
                pred_history.pop(0)

            if pred_history:
                mot_pred = max(set(pred_history), key=pred_history.count)

        mp_drawing.draw_landmarks(frame, landmarks, mp_hands.
                                HAND_CONNECTIONS)

```

```

# ===== Envoi MQTT des gestes =====
current_time = time.time()
if mot_pred != "" and mot_pred != last_sent:
    if current_time - last_time > SEND_INTERVAL:
        client.publish(TOPIC_GESTURE, mot_pred)
        last_sent = mot_pred
        last_time = current_time

# ===== Affichage =====
cv2.putText(frame, f"Geste: {mot_pred}", (10, 50),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
cv2.imshow("Reconnaissance + MQTT", frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
client.loop_stop()
client.disconnect()

```

Listing B.1 – Programme principal de traitement d’image (PC/Serveur)

Bibliographie

- [1] Google, MediaPipe Hands : Real-Time Hand Tracking, <https://developers.google.com/mediapipe>
- [2] OpenCV Documentation, <https://opencv.org>
- [3] MQTT Protocol Specification, <https://mqtt.org>
- [4] EMQX Broker, <https://www.emqx.io>
- [5] Espressif Systems, ESP32 Documentation, <https://www.espressif.com>
- [6] Adafruit SSD1306 Datasheet
- [7] Scikit-learn Documentation, <https://scikit-learn.org>
- [8] Python Documentation, <https://www.python.org>